第34课 阶段三测试卷

考试时间：50分钟 | 总分：100分

姓名：_ 日期：_

姓名：_____ 班级：_____ 学号：_____

日期：_____年_____月_____日 得分：_____ / _____

一、读程写结果（每题6分，共30分）

第1题（字符串操作）

```
#include <iostream>
#include <string>
using namespace std;

int main() {
 string s = "HelloWorld";
 cout << s.size() << endl;
 cout << s[4] << endl;
 cout << s.substr(5, 3) << endl;
 string t = s.substr(0, 5) + " " + s.substr(5, 5);
 cout << t << endl;
 return 0;
}
```

程序输出：

```
_________
_________
_________
_________
```

第2题（递归调用过程）

```
#include <iostream>
using namespace std;

int f(int n) {
 if (n <= 1) return 1;
 return f(n - 1) + f(n - 2);
}

int main() {
 cout << f(2) << endl;
 cout << f(5) << endl;
 return 0;
}
```

程序输出：

```
________
________
```

第3题（栈操作）

```
#include <iostream>
#include <stack>
using namespace std;

int main() {
 stack<int> st;
 for (int i = 1; i <= 5; i++) {
 st.push(i * 10);
 }
 st.pop();
 st.pop();
 cout << st.top() << endl;
 cout << st.size() << endl;
 return 0;
}
```

程序输出：

```
________
________
```

第4题（队列操作）

```
#include <iostream>
#include <queue>
using namespace std;

int main() {
 queue<int> q;
 q.push(3);
 q.push(7);
 q.push(1);
 q.push(9);

 q.pop();
 q.pop();

 cout << q.front() << endl;
 cout << q.back() << endl;
 cout << q.size() << endl;
 return 0;
}
```

程序输出：

```
________
________
________
```

第5题（前缀和 + vector）

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
 vector<int> a = {0, 2, 5, 1, 4, 3}; // a[0]不用，a[1]~a[5]
 vector<int> pre(6, 0);

 for (int i = 1; i <= 5; i++) {
 pre[i] = pre[i - 1] + a[i];
 }

 cout << pre[5] << endl;
 cout << pre[4] - pre[1] << endl;
 return 0;
}
```

程序输出：

```
_________
_________
```

二、改错题（每题10分，共20分）

第6题： 下面的程序想用二分查找在升序数组中找到数字 7 的位置，但有 **2处错误**，请找出并改正。

```
#include <iostream>
using namespace std;

int main() {
 int a[8] = {0, 1, 3, 5, 7, 9, 12, 15}; // a[1]~a[7]，已升序
 int target = 7;
 int left = 1, right = 7;

 while (left < right) { // 错误1在这附近
 int mid = (left + right) / 2;
 if (a[mid] == target) {
 cout << "找到了，位置是 " << mid << endl;
 break;
 } else if (a[mid] < target) {
 left = mid; // 错误2在这附近
 } else {
 right = mid - 1;
 }
 }
 return 0;
}
```

错误1： ____

改正为： ____

错误2： ____

改正为： ____

第7题： 下面的程序想按成绩从高到低排序并输出，但有 **2处错误**，请找出并改正。

```
#include <iostream>
#include <algorithm>
using namespace std;

struct Student {
 string name;
 int score;
};

bool cmp(Student a, Student b) {
 return a.score < b.score; // 错误1在这附近
}

int main() {
 Student stu[3] = {{"Tom", 80}, {"Amy", 95}, {"Bob", 70}};
 sort(stu, stu + 3, cmp);

 for (int i = 0; i <= 3; i++) { // 错误2在这附近
 cout << stu[i].name << " " << stu[i].score << endl;
 }
 return 0;
}
```

错误1： ____

改正为： ____

错误2： ____

改正为： ____

三、编程题（共50分）

第8题（15分）—— 字符串统计

题目描述：

输入一个只包含大小写字母的字符串 s（长度不超过100），统计并输出其中大写字母的个数和小写字母的个数。

输入格式：

一行一个字符串（不含空格）。

输出格式：

一行两个整数，用空格隔开，分别是大写字母个数和小写字母个数。

输入样例：

```
HelloWorld
```

输出样例：

```
2 8
```

第9题（15分）—— 括号匹配

题目描述：

输入一个只包含 `(` 和 `)` 的字符串（长度不超过100），判断括号是否完全匹配。如果匹配输出 `Yes`，否则输出 `No`。

说明： 括号匹配的意思是每个左括号 `(` 都有一个对应的右括号 `)` 与之配对，且顺序正确。

输入格式：

一行一个括号字符串。

输出格式：

一行，输出 `Yes` 或 `No`。

输入样例1：

```
(()())
```

输出样例1：

```
Yes
```

输入样例2：

```
(()
```

输出样例2：

```
No
```

第10题（20分）—— 成绩排名

题目描述：

有 n 个同学（1 ≤ n ≤ 100），每个同学有姓名和成绩。请按成绩从高到低排序输出所有同学的姓名和成绩。如果成绩相同，保持输入时的先后顺序。

输入格式：

第一行一个整数 n。
接下来 n 行，每行一个字符串（姓名，无空格）和一个整数（成绩，0~100）。

输出格式：

按成绩从高到低，输出 n 行，每行输出姓名和成绩，用空格隔开。

输入样例：

```
4
Alice 88
Bob 95
Cindy 88
David 72
```

输出样例：

```
Bob 95
Alice 88
Cindy 88
David 72
```